



LaurentianUniversity
Université**Laurentienne**

BHARTI SCHOOL OF ENGINEERING AND COMPUTER SCIENCE

Design and Automate an IoT-Based Smart Parking Space Monitoring System for Laurentian University's Main Parking Lot using Tinkercad

CPSC-5207EL-01 Internet of Things (IoT)

Faculty Advisor:
Dr. Williams Nwadiugwu

Spring, 2026.

Submitted By - GROUP 25

Name	Student ID
Zahidul Islam	0472271
Md. Hasibul Islam	0478178
Kh Md Nazmul Hossain	0475850

27 June 2026

Chapter 1 – Project Synopsis, and Literature Review.	3
1.1. Project Synopsis:	3
Chapter 2 – System Model.	7
2.1. Technology Stack and System Requirements:	7
2.2. DFDs, Class Diagrams and Entity Relationships (ERs):	9
Chapter 3 – Implementation, Results and Conclusion.	12
3.1. System Methods/Functions Implementation:.....	12
3.2. System Results:.....	20
References	26

Chapter 1 – Project Synopsis, and Literature Review.

1.1. Project Synopsis:

- **Project Abstract:** This project demonstrates an IoT-based smart parking space monitoring system designed for Laurentian University's main parking lot using the Tinkercad simulation environment. The prototype uses two Arduino Uno boards, seven 3-pin ultrasonic distance sensors, two servo motors, one PIR motion sensor, one 4x4 keypad, a 16x2 LCD, a NeoPixel strip, and a piezo buzzer. The system monitors parking-slot availability detects obstacles from left, front, and right directions, controls entry and exit gate movement, and displays real time parking availability.

The project inherits the smart-city idea of using sensors, microcontrollers, automation, and real-time feedback to improve urban services. The Internet of Things connects physical devices with sensors and software so that they can collect and exchange useful data [1]. In this simulation, ultrasonic sensors provide information about real-time parking and obstacles. Arduino boards process the inputs, and output devices such as the LCD, NeoPixel strip, buzzer, and servo gates to respond automatically. The system has a two-node Arduino design. Keypad authentication, PIR-based vehicle movement detection, and servo gate operation are managed by a gate access node. While the parking monitoring node reads the parking-slot sensors and obstacle sensors, calculates availability, updates indicators, activates the buzzer, and prints system status through the Serial Monitor as a simulated IoT dashboard.

- **Literature Review:** Smart parking systems are a practical application of IoT in smart cities because they convert parking spaces into measurable real-time data points. Earlier smart parking research has shown that real-time allocation and reservation can reduce the time required to find parking and improve the use of limited parking capacity [2]. Modern smart parking systems often combine sensor data, reservation logic, real-time databases, and user-facing dashboards or mobile applications [3]. A deployed parking sensing system at Stanford University demonstrated a complete smart parking information pipeline that includes sensor deployment, cloud service, real-time processing, and visualization of occupancy information [4]. This supports the design direction of the current prototype: the system does not only open a gate; it also monitors parking availability and produces usable status information.

The current project uses Tinkercad for simulation since it supports virtual electronics design and Arduino code simulation, which makes it suitable for testing sensor and actuator logic before building physical hardware [5]. Arduino Uno provides digital I/O, PWM pins, analog inputs, USB programming, and enough capability for small, embedded prototypes [6]. This prototype uses 3-pin ultrasonic distance sensors with VCC, GND, and SIG connections. These sensors support a PING-style ultrasonic interface where a single signal wire is used for both transmitting the ultrasonic pulse and receiving the echo response [7]. The Arduino sketch includes timeout handling to avoid indefinite waiting and measures the returned pulse duration using `pulseIn()` [8].

- **Projects vision:** The vision is to design a small-scale but realistic smart parking prototype that demonstrates how a university parking lot can become part of a smart-city system. The system aims to reduce the time drivers spend searching for parking, improve visibility of available spaces, reduce unnecessary vehicle movement, and automate basic gate-control tasks.

In a physical deployment, a similar concept can be used by replacing or supplementing the simulated Serial Monitor output with a Wi-Fi-enabled microcontroller, cloud database, mobile application, and administrative dashboard. This would allow students, staff, and visitors to check availability before reaching the parking area and would allow the university to analyze parking usage over time.

- **Objectives:** The objectives of this simulation project are to design an IoT-based smart parking system that can detect parking availability for four parking slots and identify nearby obstacles from the left, front, and right directions using ultrasonic distance sensors. The system aims to use a buzzer that activates each time an obstacle is detected within the safety threshold, display available number of parking slots in a 16.2 LCD, and indicate parking space availability with NeoPixel strip. The prototype will use two password-controlled servo motors to simulate entrance and exit gates. A keypad will allow users to enter passwords, and a PIR sensor will send motion detection to rotate the servo gates once password access is granted. The system also aims to print parking status, sensor readings, access events, and gate activity

to the Serial Monitor to simulate an IoT dashboard output. Finally, the full circuit and code behavior will be validated in the Tinkercad simulation environment.

- **System Flowchart:** The system consists of 2 Arduino controllers, managing 3 different flows. The gate access controller manages the entrance, exit and password validation flow, while the parking monitoring node calculates and displays parking availability, activates buzzer, and generates useful analytics in serial monitor.

Flowchart parking monitoring node:

Step:1 **Start**
The smart parking monitoring system begins.

Step:2 **Initialize** **System**
All major components are initialized, including the LCD display, NeoPixel strip, 3-pin ultrasonic distance sensors, keypad, PIR sensor, buzzer, servo motors, and Serial Monitor.

Step:3 **Read** **Parking** **Slot** **Sensors**
The Arduino reads the four parking-slot ultrasonic sensors to measure the distance in each parking space.

Step:4 **Calculate** **Parking** **Availability**
The system compares each parking-slot distance with the parking threshold. If the measured distance is within the occupied range, the slot is marked as occupied. Otherwise, it is marked as available.

Step:5 **Update** **LCD** **Display**
The LCD displays the current number of available parking spaces.

Step:6 **Update** **NeoPixel** **Status**
The NeoPixel strip updates the parking status visually. Green indicates that parking spaces are available, while red indicates that the parking lot is full.

Step:7 **Read** **Obstacle** **Sensors**
The system reads the left, front, and right ultrasonic distance sensors to detect nearby obstacles.

Step:8 **Check** **Obstacle** **Distance**
If any obstacle is detected within the defined threshold, the buzzer turns on and the obstacle direction is printed in the Serial Monitor. If no obstacle is detected, the buzzer remains off and the system continues monitoring.

Step:9 **Check** **Keypad** **Input**
The system checks whether the user has entered a keypad command.

Step:10 **Keypad** **Command** **Decision**
If no keypad command is entered, the system continues monitoring the parking slots and obstacle sensors. If a keypad command is entered, the system moves to the gate to access flow.

Step:11 Print Serial Monitor Status: The system prints the latest parking-slot distance, available slot count, obstacle status, and system state to the Serial Monitor.

Step:12 **Loop** **Back** **to** **Monitoring**
The system returns to reading parking-slot sensors and repeats the monitoring cycle continuously.

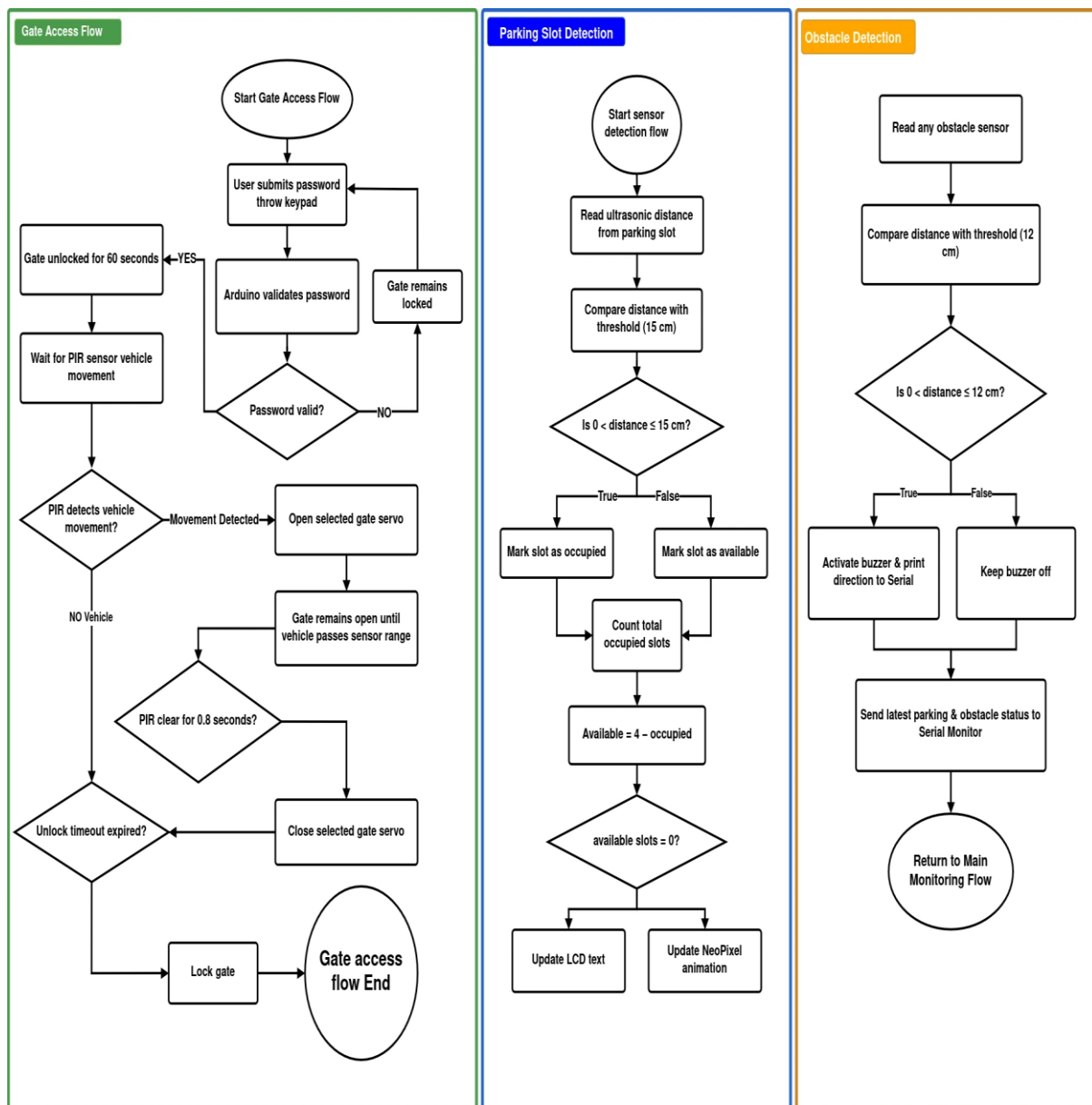


Figure 1.1: Smart parking Flowchart

Entry and Exit Gate Access Flowchart

Step:1	Start	Gate	Access	Flow
	The gate access process begins when the user enters a keypad command.			
Step:2	Read	Keypad	Command	
	The Arduino reads the command entered through the 4x4 keypad.			
Step:3	Check	Command	Type	
	The system identifies whether the user is requesting entrance or exit access. A1234# is used for entrance access, and B1234# is used for exit access.			
Step:4	Validate	Command	Format	
	The system checks whether the command follows the correct format.			
Step:5	Command	Format	Decision	
	If the command format is incorrect, the system denies access and returns to the main monitoring flow. If the command format is correct, the system continues to password verification.			

Step:6	Check	Password
The system compares the entered password with the stored password.		
Step:7	Password	Decision
If the password is wrong, access is denied, the gate remains closed, and the event is printed in the Serial Monitor. If the password is correct, the selected gate is unlocked for a limited time.		
Step:8	Unlock	Selected Gate
The system unlocks either the entrance gate or exit gate depending on the keypad password.		
Step:9	Wait for PIR	Detection
The PIR sensor waits for vehicle movement near the gate.		
Step:10	PIR Detection	Decision
If the PIR sensor detects vehicle movement, the selected servo motor opens the gate. If no movement is detected, the system continues waiting until the unlock timer expires.		
Step:11	Timeout	Decision
If the unlock time expires before vehicle movement is detected, the gate is locked again, and the system returns to the main monitoring flow. If the timeout has not expired, the system keeps waiting for PIR detection.		
Step:12	Open	Selected Gate
The selected servo motor rotates to open the entrance or exit gate.		
Step:13	Keep the Gate Open During Movement	
The gate remains open while the PIR sensor continues detecting vehicle movement.		
Step:14	Check PIR	Clear State
The system checks whether the PIR sensor has remained clear for the required delay period.		
Step:15	Close	Gate
When the PIR sensor is clear, the selected servo motor closes the gate.		
Step:16	Return to Main Monitoring	Flow
After the gate closes, the system returns to the main monitoring flow and continues checking parking availability and obstacle status.		

Chapter 2 – System Model.

2.1. Technology Stack and System Requirements:

- **Project requirements:** The project requires a smart parking space monitoring system for Laurentian University's main parking lot using Tinkercad. The system must target at least four parking slots, including right, left, and front ultrasonic sensors for obstacle detection, and activate buzzer alerts when a configured distance threshold is reached. The system should also include working Arduino source code, circuit design, screenshots of operation, and testing/validation results.

The completed prototype meets these requirements through a two-Arduino circuit. One Arduino handles secure gate access and gate movement, while the second Arduino handles parking occupancy monitoring, directional obstacle detection, display updates, NeoPixel status indication, buzzer alerts, and Serial Monitor reporting.

- **Technologies / Tools Stack:**
The two-board approach here helps avoid I/O pin crowding. The gate access board uses keypad row/column pins, two servo pins, one PIR input, and one gate signal output. The parking monitoring board uses seven ultrasonic signal pins, one buzzer pin, one gate signal input, and the display/NeoPixel pins. Each ultrasonic sensor uses one SIG pin where the code switches between output mode and input

mode during distance measurement. The following table describes each component, quantity, and purpose in the prototype.

Component / Tool	Quantity	Purpose
Arduino Uno	2	Microcontroller boards for gate access and parking monitoring [6]
PING-style ultrasonic distance sensor	7	Four parking-slot sensors and three directional obstacle sensors using one SIG pin per sensor [7], [8]
Servo motor	2	Entrance and exit gate movement; controlled using Servo library [9]
PIR sensor	1	Vehicle movement detection near gate area
4x4 keypad	1	Password-based command input; uses matrix keypad scanning [10]
16x2 LCD	1	Displays available parking count [11]
NeoPixel strip	1	Visual status indication for parking and gate activity [12]
Piezo buzzer	1	Audible obstacle alert
Breadboard and jumper wires	As needed	Circuit wiring and common power/ground distribution
Tinkercad Circuits	1 platform	Virtual circuit simulation and Arduino code testing [5]
Arduino C/C++	2 sketches	Embedded logic for both Arduino nodes

● **Project scheduling and Tasks/Function Distribution:**

Date	LEAD I: Project Design and Code Implementation (Due: JUNE 27, 2026) Member's First Name: Zahidul	LEAD II: Project Design and Code Implementation (Due: JUNE 27, 2026) Member's First Name: Md. Hasibul	LEAD III: Documentation, Reports and Presentation (Due: JUNE 27, 2026) Member's First Name: Kh Md Nazmul
Wed, June 03 rd	- Assign roles and responsibilities to each team member. - Research the operational mechanics of smart parking systems. - Study how ultrasonic sensors detect distance and measure proximity. - Determine the configuration and placement of the left, right, and forward sensors to effectively monitor the parking space. - Develop the foundational Arduino code for the system.	- Ensure to detail all your activities in clearly defined bullet points or numerical order, like the one used here. - Make them clear and concise. - Assign specific roles to each team member. - Set up the Tinkercad workspace for the project. - Select and add all required components for the IoT smart parking system circuit. - Establish the electrical connections between the components. - Design and finalize the initial circuit layout.	- Assign specific roles to each team member. - Establish a weekly progress report schedule to track updates. - Compile and organize the project documentation. - Begin drafting the final report and designing the presentation slides.

Wed, June 10 th	• Measure the distance of an ultrasonic sensor by Arduino program. • Worked on the logic to identify the presence of a vehicle based on threshold distance. • Implemented the logic of LED to indicate parking spot availability. • Worked on the code of parking gate automation using the servo motor. • Debug, and test the application code to confirm the system's functionalities.	• Assembled the complete circuit design in the Tinkercad simulation environment. • Configured the TRIG and ECHO pins to connect the ultrasonic sensor to the Arduino. • Configured the LED to provide the visual status indication. • Integrated the Servo motor with Arduino to simulate gate automation. • Tested the simulation and confirmed all components were functioning as expected.	• Updated the Week 2 progress report with the tasks completed and milestones achieved. • The system implementation process and its overall operation are documented. • Added screenshots of the circuit design and simulation results to support the project documentation. • LED, Sensor and servo motor functionality described. • Started preparing and organizing content that will be included in the final project report.
Wed, June 17 th	• Latest weekly progress must be added/ appended to existing file to show continuity of weekly activities. • Continue to update your weekly activities in the same single document. • No two or more documents are allowed. • Expanded the Arduino setup to track multiple parking spots at once using a handful of ultrasonic sensors. • Built out the logic to monitor each spot in real time so the system always knows what's open and what's taken. • Set up live feeds via the serial monitor to give quick, readable updates on parking availability. • Fine-tuned the code to sharpen sensor accuracy and weed out any false readings.	• Scaled up the circuit by wiring in extra ultrasonic sensors to simulate a multi-space parking lot. • Organized and refined wiring connections for better circuit clarity. • Ran the complete system through its paces in Tinkercad to make sure it handles all the sensor inputs smoothly. • Double-checked that the LEDs for each individual parking spot are lighting up exactly when they're supposed to.	• Updated week 3 progress report, making sure to log all the new system upgrades. • Added explanation of multi- slot parking logic and working principle. • Captured screenshots of the updated circuit layout directly from Tinkercad. • Detailed exactly how the LEDs, sensors, and servo motors all play their parts in the system. • Started drafting system architecture diagrams for the final report.

Wed, June 24 th	• Visited and collected information about the Parking spaces and use cases of visitors and employees parking lot. • Added a second Arduino with a keypad, PIR sensor and two servo gate in the circuit for parking gate simulation. • Wired the keypad pins, PIR sensor pins and servo pins with Arduino 2. • Write code to simulate a lock and unlock state with keypad password entry, configured A key to trigger entrance gate and B key to trigger exit gate unlock. • Programmed the PIR sensor distance detection to open the servo gates when in unlock state. • For quick readable updates and troubleshooting issues added live feed in serial monitor.	• Collected information from the parking lot of Fraser building to improve the idea and design of the group project. • Wired and programmed 3 more ultrasonic sensors in the main circuit to simulate left, right and front obstacle detection. • Wired and programmed a buzzer to generate alert when obstacle sensors detect collision. • Wired and programmed a NeoPixel Strip 4 to show green and red light simulating parking occupied or available. • Connected Gate controlling arduino with parking monitoring arduino to receive servo gate open or close signals and blink the NeoPixel Strip when it happens.	• Read documentation on Arduino, Tinkercad, Adafruit library and an IEEE report of smart parking based on resource allocation. • Sat with the group to understand the finalized circuit and code. • Completed the draft report with references and citations. • Prepared DFD 0, DFD 1, System architecture diagram, Gate control and parking work flow diagram. • Prepared ERD and class diagram for final report. • Included all the diagrams in the presentation slides. • Prepared script for circuit video demonstration.
----------------------------	--	--	---

2.2. DFDs, Class Diagrams and Entity Relationships (ERs):

- **Project Data Flow Diagrams (DFDs):** The Level 0 Data Flow Diagram shows the smart parking system as one main process. The driver or vehicle provides input to the system through keypad commands, vehicle presence, and movement detection. The smart parking monitoring and gate access system processes these inputs to determine parking availability, obstacle status, and gate operation.

The system sends parking availability, status updates, and obstacle alerts to the LCD display, NeoPixel status indicator, and piezo buzzer. It also sends gate open/close commands to the entrance and exit servo motors. At the same time, distance readings, parking count, system status, and access events are sent to the Serial Monitor, which acts as a simulated IoT dashboard. This diagram gives a high-level view of how the user, sensors, output devices, gate system, and monitoring interface interact with the main smart parking process.

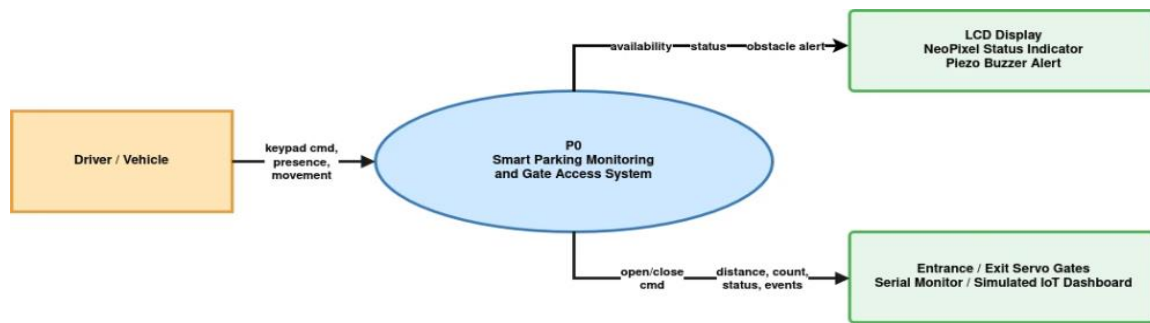


Figure 2.2: Level 0 data flow diagram of the smart parking system.

The Level 1 Data Flow Diagram shows the internal processing modules of the smart parking system. External inputs such as parking-slot ultrasonic sensors, left/front/right obstacle sensors, keypad commands, and PIR motion signals are received by the Arduino processes. The system first reads the parking-slot sensor distances and calculates the number of available spaces. It also reads the directional obstacle sensors and checks whether any obstacle is within the defined threshold distance.

The keypad authentication process validates the user's command and password before allowing entrance or exit access. If access is granted, the gate control process operates the entrance or exit servo motor using the PIR motion signal. The display and indicator process updates the LCD and NeoPixel strip with the current parking status, while the buzzer is activated when an obstacle alert is detected. The Serial Monitor process works as a simulated IoT dashboard by showing parking availability, sensor readings, gate status, and access events.

The diagram also includes data stores such as current parking state, access event log, and sensor reading buffer. These represent how the system stores or tracks important status information during operation.

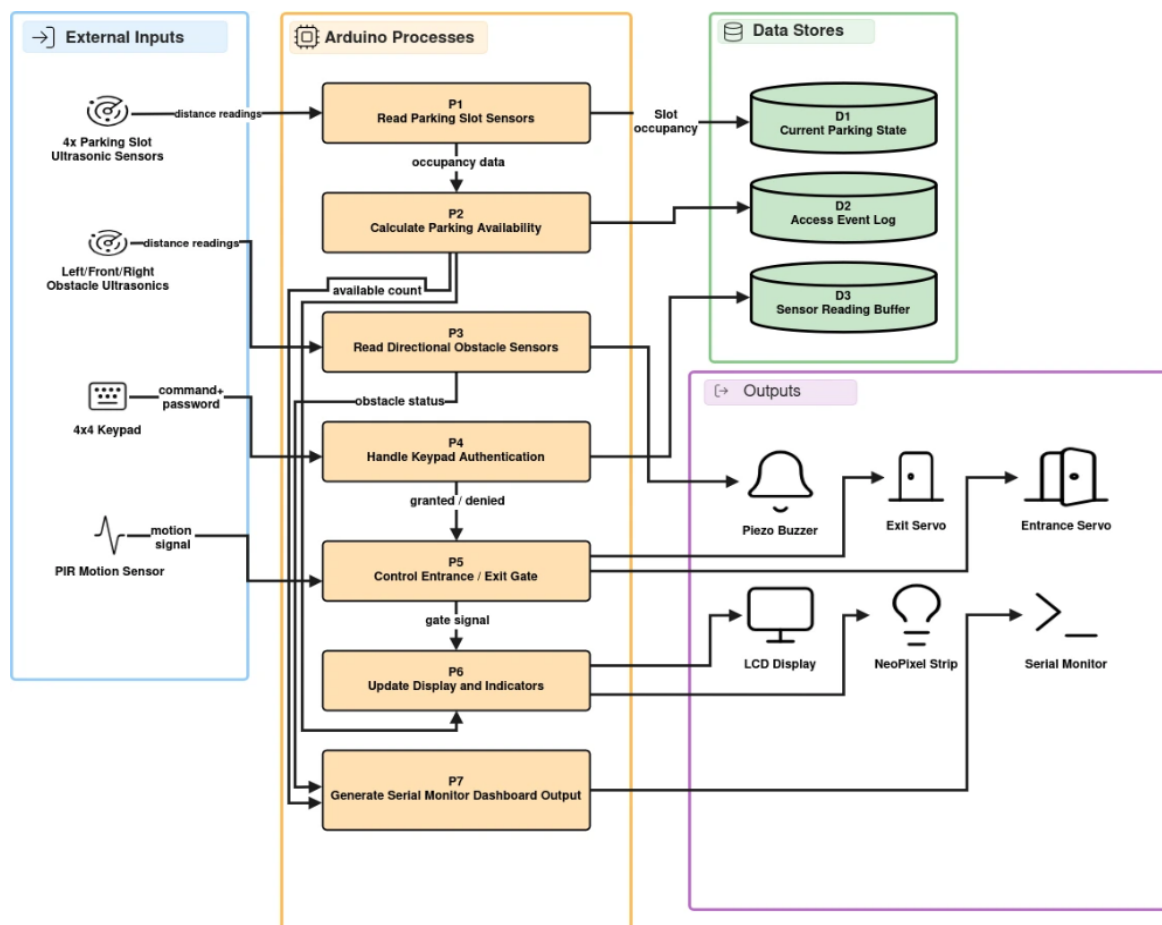
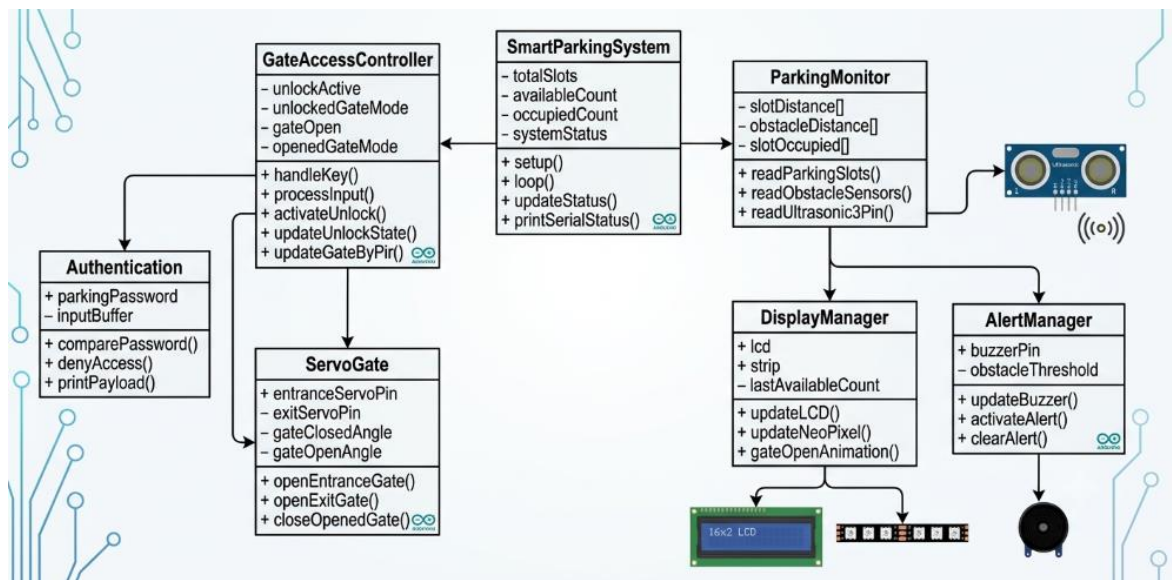


Figure 2.3: Level 1 data flow diagram showing sensor, processing, and output module integration.

- **Project Class Diagrams:** The Class / Module Diagram shows the logical structure of the smart parking system. As it is built using Arduino C/C++, it highlights functional modules rather than strict object-oriented classes.
 - The **SmartParkingSystem** module acts as the central controller, managing slot counts, system status, and core functions like setup(), loop(), and status updates.
 - The **GateAccessController** handles entrance and exit logic, processing keypad input, managing gate states, and controlling access based on PIR detection.
 - The **Authentication** module verifies passwords, manages keypad input, and logs access events.
 - The **ServoGate** module controls the servo motors for opening and closing entrance and exit gates.
 - The **ParkingMonitor** reads ultrasonic sensors to track slot occupancy and detect obstacles.
 - The **DisplayManager** updates the LCD, NeoPixel strip, and animations based on system status.
 - The **AlertManager** triggers buzzer alerts when obstacles are detected within a threshold.

Overall, the system is organized into modules for monitoring, access control, display, and alerts, making it easier to manage and extend.

**Figure 2.4:** Class/module diagram of the smart parking software functions.

- **Project ERs:** The Entity Relationship Diagram shows the conceptual data model of the smart parking system and how it could be structured for a cloud-connected IoT platform.

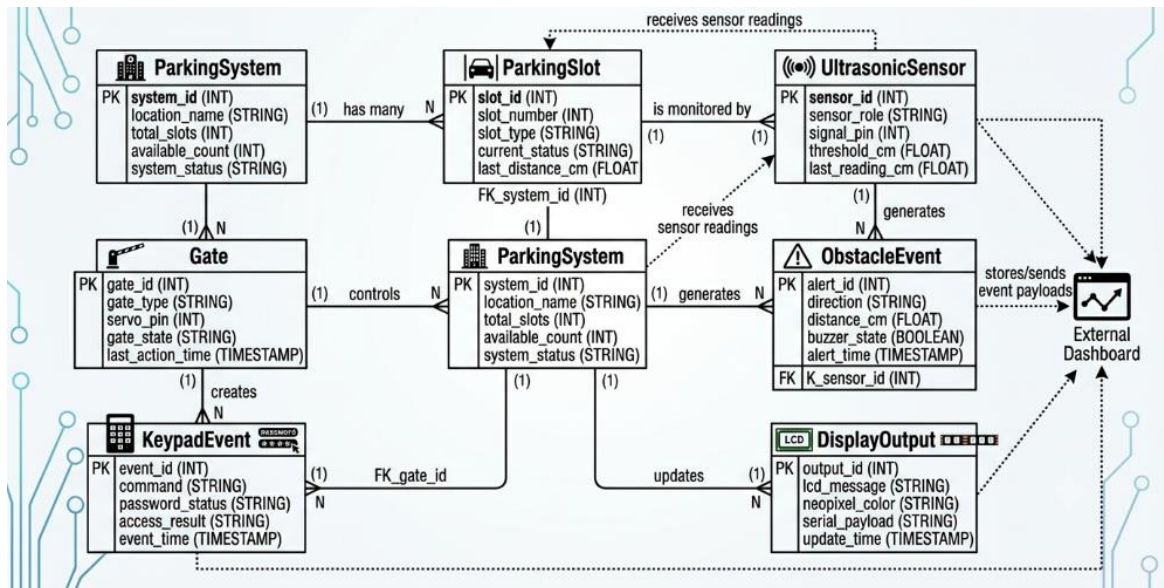


Figure 2.5: Entity relationship diagram of the smart parking system data model.

- The **ParkingSystem** entity stores system details such as ID, location, total slots, available slots, and status. It is linked to multiple parking slots and gates.
- The **ParkingSlot** entity represents each parking space, including its ID, type, status, and distance reading, monitored by an ultrasonic sensor.
- The **UltrasonicSensor** entity stores sensor details such as ID, role, pin, threshold, and last reading, used for slot monitoring or obstacle detection.
- The Gate entity represents entrance and exit gates, storing gate ID, type, servo pin, state, and last action time. Each gate can generate multiple keypad events.
- The **KeypadEvent** entity records access attempts, including command, password status, result, and time.
- The **ObstacleEvent** entity logs obstacle alerts with direction, distance, buzzer state, and time.
- The **DisplayOutput** entity stores output data such as LCD messages, NeoPixel color, serial payload, and update time.

An external dashboard can receive this data for real-time monitoring. Overall, the ER diagram shows how system components and data are connected.

Chapter 3 – Implementation, Results and Conclusion.

3.1. System Methods/Functions Implementation:

- **All Methods/Functions Implementation:** The implementation is split into two Arduino sketches. The gate access sketch manages keypad authentication, unlock state, PIR detection, servo control, and event payload printing. The parking monitoring sketch manages ultrasonic distance readings, parking availability calculation, obstacle alert detection, LCD updates, NeoPixel updates, buzzer activation, and Serial Monitor output.

Gate Access Function	Purpose
setup()	Initializes Serial Monitor, two servos, gate signal pin, PIR pin, and default closed gate state.
loop()	Continuously checks keypad input, unlock timeout, and PIR-based gate movement.
handleKey()	Reads keypad characters, appends them to the input buffer, clears input on *, and submits on #.
processInput()	Validates command format, checks A/B gate mode, compares password, and grants or denies access.

activateUnlock()	Creates one-minute unlock state after successful password input.
updateUnlockState()	Cancels unlock if no vehicle movement occurs within the unlock duration.
updateGateByPir()	Opens selected gate when PIR detects motion and closes the gate after PIR remains clear.
openEntranceGate()	Moves entrance servo to open angle and sends gate signal HIGH.
openExitGate()	Moves exit servo to open angle and sends gate signal HIGH.
closeOpenedGate()	Closes the active gate and sends gate signal LOW.
denyAccess()	Prints access-denied reason to Serial Monitor.
printPayload()	Prints simulated IoT event data as a JSON-like payload.

Parking Monitoring Function	Purpose
setup()	Initializes Serial Monitor, LCD, NeoPixel strip, buzzer pin, gate signal input, and starting display.
loop()	Reads slots and obstacles, updates outputs, and prints periodic status.
readParkingSlots()	Reads four single-SIG ultrasonic slot sensors and counts occupied spaces.
readObstacleSensors()	Reads left, front, and right single-SIG ultrasonic obstacle sensors and returns closest alert direction.
readUltrasonic3Pin()	Uses one SIG pin: sends the ultrasonic pulse as OUTPUT, switches the same pin to INPUT, measures echo duration with pulseIn(), and converts it to distance.
updateLCD()	Displays only the current available parking count.
updateNeoPixel()	Shows green when parking is available and red when the lot is full.
gateOpenAnimation()	Runs NeoPixel color animation while the gate signal is HIGH.
setAllPixels()	Applies one RGB color to all NeoPixel LEDs.
updateBuzzer()	Sounds buzzer when an obstacle alert is active.
printSerialStatus()	Prints slot distances, availability, obstacle distances, and status output.

● **Project Methods/Functions code snippets:** The following code excerpts show the core logic of the two sketches. Full source code should be submitted separately as .ino files in the final zipped project folder.

Gate Access Node - Constants and Setup

```
#include <Keypad.h>
#include <Servo.h>

// ----- Servos -----
Servo entranceServo;
Servo exitServo;

const int entranceServoPin = 10;
const int exitServoPin = 11;

const int gateClosedAngle = 0;
const int gateOpenAngle = 90;

// Signal to Parking Monitoring Arduino
// Gate Arduino D12 -> Parking Arduino D11
const int gateSignalPin = 12;

// ----- One PIR Sensor for Both Gates -----
const int pirPin = A0;

// ----- Timing -----
const unsigned long unlockDuration = 60000;    // 1 minute unlock state
const unsigned long clearConfirmTime = 800;    // PIR must be LOW for 0.8 sec before closing

// ----- Keypad -----
const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'0', '*', '#', 'D'}
```

```

    {'*', '0', '#', 'D'}
};

byte rowPins[ROWS] = {2, 3, 4, 5};
byte colPins[COLS] = {6, 7, 8, 9};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

// ----- Fixed Password -----
const String parkingPassword = "1234";

// ----- Input -----
String inputBuffer = "";

// ----- Unlock/Gate State -----
bool unlockActive = false;
char unlockedGateMode = 'N'; // A = entrance, B = exit, N = none
unsigned long unlockStartedAt = 0;

bool gateOpen = false;
char openedGateMode = 'N';
unsigned long clearStartedAt = 0;
void setup() {
    Serial.begin(9600);

    entranceServo.attach(entranceServoPin);
    exitServo.attach(exitServoPin);
    entranceServo.write(gateClosedAngle);
    exitServo.write(gateClosedAngle);
    pinMode(gateSignalPin, OUTPUT);
    digitalWrite(gateSignalPin, LOW);

    pinMode(pirPin, INPUT);

    Serial.println("=====");
    Serial.println("Gate Access Node Started");
    Serial.println("=====");
    Serial.println("Fixed password: 1234");
    Serial.println();
    Serial.println("A1234# = Unlock entrance gate for 1 minute");
    Serial.println("B1234# = Unlock exit gate for 1 minute");
    Serial.println("PIR motion during unlock state opens selected gate");
    Serial.println("* = Clear input");
    Serial.println("-----");
}

```

Gate Access Node - Main Loop and Key Handling

```

void loop() {
    char key = keypad.getKey();

    if (key) {
        handleKey(key);
    }

    updateUnlockState();
    updateGateByPir();
}

void handleKey(char key) {
    Serial.print("Key pressed: ");
    Serial.println(key);

    if (key == '*') {
        inputBuffer = "";
        Serial.println("Input cleared");
        return;
    }

    if (key == '#') {
        processInput(inputBuffer);
        inputBuffer = "";
        return;
    }

    if (inputBuffer.length() < 5) {
        inputBuffer += key;
        Serial.print("Current input: ");
        Serial.println(inputBuffer);
    }
}

```

```

    } else {
        Serial.println("Input too long. Press * to clear.");
    }
}

```

Gate Access Node - Input Processing and Unlock Logic

```

// ----- Input Processing -----
// Expected format: A1234 or B1234
void processInput(String command) {
    Serial.print("Submitted command: ");
    Serial.println(command);

    if (command.length() != 5) {
        denyAccess("Invalid input. Use A1234# or B1234#");
        return;
    }

    char mode = command.charAt(0);
    String enteredPassword = command.substring(1, 5);

    if (mode != 'A' && mode != 'B') {
        denyAccess("First key must be A for entry or B for exit");
        return;
    }

    if (enteredPassword != parkingPassword) {
        denyAccess("Wrong password");
        printPayload(mode, "DENIED", "WRONG_PASSWORD");
        return;
    }

    activateUnlock(mode);
}

// ----- Unlock State -----
void activateUnlock(char mode) {
    unlockActive = true;
    unlockedGateMode = mode;
    unlockStartedAt = millis();

    if (mode == 'A') {
        Serial.println("Correct password. ENTRANCE gate unlocked for 1 minute.");
        printPayload(mode, "GRANTED", "ENTRANCE_UNLOCKED");
    } else {
        Serial.println("Correct password. EXIT gate unlocked for 1 minute.");
        printPayload(mode, "GRANTED", "EXIT_UNLOCKED");
    }

    Serial.println("Waiting for PIR motion near gate...");
}

void updateUnlockState() {
    if (!unlockActive) {
        return;
    }

    if (!gateOpen && millis() - unlockStartedAt >= unlockDuration) {
        unlockActive = false;
        unlockedGateMode = 'N';

        Serial.println("Unlock time expired. Gate locked again.");
    }
}

```

Gate Access Node - PIR and Servo Gate Control

```

// ----- PIR Gate Logic -----
void updateGateByPir() {
    int pirState = digitalRead(pirPin);

    // If unlocked and gate is closed, PIR motion opens the selected gate
    if (unlockActive && !gateOpen && pirState == HIGH) {
        if (unlockedGateMode == 'A') {
            openEntranceGate();
        } else if (unlockedGateMode == 'B') {
            openExitGate();
        }
    }

    // If gate is open, close only after PIR becomes LOW
}

```

```

    if (gateOpen) {
        if (pirState == HIGH) {
            clearStartedAt = 0;
        } else {
            if (clearStartedAt == 0) {
                clearStartedAt = millis();
            }

            if (millis() - clearStartedAt >= clearConfirmTime) {
                closeOpenedGate();
            }
        }
    }
}

// ----- Gate Open -----
void openEntranceGate() {
    Serial.println("PIR detected motion. Opening ENTRANCE gate...");

    entranceServo.write(gateOpenAngle);
    digitalWrite(gateSignalPin, HIGH);

    gateOpen = true;
    openedGateMode = 'A';
    clearStartedAt = 0;

    printPayload('A', "GRANTED", "ENTRANCE_GATE_OPENED");
}

void openExitGate() {
    Serial.println("PIR detected motion. Opening EXIT gate...");

    exitServo.write(gateOpenAngle);
    digitalWrite(gateSignalPin, HIGH);

    gateOpen = true;
    openedGateMode = 'B';
    clearStartedAt = 0;

    printPayload('B', "GRANTED", "EXIT_GATE_OPENED");
}

// ----- Gate Close -----
void closeOpenedGate() {
    if (openedGateMode == 'A') {
        entranceServo.write(gateClosedAngle);
        Serial.println("PIR clear. ENTRANCE gate closed.");
        printPayload('A', "CLOSED", "ENTRANCE_GATE_CLOSED");
    } else if (openedGateMode == 'B') {
        exitServo.write(gateClosedAngle);
        Serial.println("PIR clear. EXIT gate closed.");
        printPayload('B', "CLOSED", "EXIT_GATE_CLOSED");
    }

    digitalWrite(gateSignalPin, LOW);

    gateOpen = false;
    openedGateMode = 'N';
    clearStartedAt = 0;
}

// ----- Access Denied -----
void denyAccess(String reason) {
    Serial.print("Access denied: ");
    Serial.println(reason);
}

```

Parking Monitoring Node - Constants and Setup

```

#include <Adafruit_LiquidCrystal.h>
#include <Adafruit_NeoPixel.h>

Adafruit_LiquidCrystal lcd(0);

#define NEOPIXEL_PIN 10
#define NUM_PIXELS 4

Adafruit_NeoPixel strip(NUM_PIXELS, NEOPIXEL_PIN, NEO_GRB + NEO_KHZ800);

// Parking sensors
const int totalSlots = 4;
int slotPins[4] = {2, 3, 4, 5};

```

```

// Obstacle sensors
const int totalObstacleSensors = 3;
int obstaclePins[3] = {6, 7, 8};

const int buzzerPin = 9;
const int gateSignalPin = 11;

// Thresholds
const int parkingThreshold = 15;    // cm
const int obstacleThreshold = 12;   // cm

// Data
long slotDistance[4];
long obstacleDistance[3];
bool slotOccupied[4];

unsigned long lastSerialUpdate = 0;
int lastAvailableCount = -1;

void setup() {
    Serial.begin(9600);

    lcd.begin(16, 2);
    lcd.setBacklight(1);

    strip.begin();
    strip.setBrightness(80);
    strip.clear();
    strip.show();

    pinMode(buzzerPin, OUTPUT);
    noTone(buzzerPin);

    pinMode(gateSignalPin, INPUT);

    setAllPixels(0, 255, 0);
    strip.show();

    showWelcomeMessage();

    lastAvailableCount = -1;
    updateLCD(totalSlots);

    Serial.println("Parking Monitoring Node Started");

    delay(1000);
}

// Show welcome message
void showWelcomeMessage() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Welcome to LU");

    lcd.setCursor(0, 1);
    lcd.print("Smart Parking");

    delay(2500);

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("System Starting");

    lcd.setCursor(0, 1);
    lcd.print("Please Wait...");

    delay(1500);
}

```

Parking Monitoring Node - Main Loop

```

void loop() {
    int occupiedCount = readParkingSlots();
    int availableCount = totalSlots - occupiedCount;

    int obstacleAlert = readObstacleSensors();

    if (digitalRead(gateSignalPin) == HIGH) {
        gateOpenAnimation();
    } else {

```

```

    updateNeoPixel(availableCount);
}

updateLCD(availableCount);
updateBuzzer(obstacleAlert);

if (millis() - lastSerialUpdate >= 2000) {
    printSerialStatus(occupiedCount, availableCount, obstacleAlert);
    lastSerialUpdate = millis();
}

delay(200);
}

```

Parking Monitoring Node - Parking and Obstacle Reading

```

// ----- Read Parking Sensors -----
int readParkingSlots() {
    int occupied = 0;

    for (int i = 0; i < totalSlots; i++) {
        slotDistance[i] = readUltrasonic3Pin(slotPins[i]);

        if (slotDistance[i] > 0 && slotDistance[i] <= parkingThreshold) {
            slotOccupied[i] = true;
            occupied++;
        } else {
            slotOccupied[i] = false;
        }

        delay(30);
    }

    return occupied;
}

// ----- Read Obstacle Sensors -----
int readObstacleSensors() {
    int alertIndex = -1;
    long closestDistance = 9999;

    for (int i = 0; i < totalObstacleSensors; i++) {
        obstacleDistance[i] = readUltrasonic3Pin(obstaclePins[i]);

        if (obstacleDistance[i] > 0 && obstacleDistance[i] <= obstacleThreshold) {
            if (obstacleDistance[i] < closestDistance) {
                closestDistance = obstacleDistance[i];
                alertIndex = i;
            }
        }

        delay(30);
    }

    return alertIndex;
}

// ----- 3-pin Ultrasonic Sensor -----
long readUltrasonic3Pin(int sigPin) {
    pinMode(sigPin, OUTPUT);

    digitalWrite(sigPin, LOW);
    delayMicroseconds(2);

    digitalWrite(sigPin, HIGH);
    delayMicroseconds(10);

    digitalWrite(sigPin, LOW);

    pinMode(sigPin, INPUT);

    long duration = pulseIn(sigPin, HIGH, 30000);

    if (duration == 0) {
        return -1;
    }

    long distance = duration * 0.034 / 2;
    return distance;
}

```

Parking Monitoring Node - Display, NeoPixel, Buzzer, and Serial Output

```
// ----- LCD: Only available parking count -----
void updateLCD(int availableCount) {
    if (availableCount == lastAvailableCount) {
        return;
    }

    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("Slots Available:");

    lcd.setCursor(0, 1);
    lcd.print(availableCount);

    lastAvailableCount = availableCount;
}

// ----- NeoPixel Normal Parking Status -----
void updateNeoPixel(int availableCount) {
    if (availableCount > 0) {
        setAllPixels(0, 255, 0);      // Green = parking available
    } else {
        setAllPixels(255, 0, 0);      // Red = no parking available
    }

    strip.show();
}

// ----- NeoPixel Gate Animation -----
void gateOpenAnimation() {
    setAllPixels(255, 0, 0);          // Red
    strip.show();
    delay(250);

    setAllPixels(0, 255, 0);          // Green
    strip.show();
    delay(250);

    setAllPixels(0, 0, 255);          // Blue
    strip.show();
    delay(250);

    setAllPixels(255, 180, 0);        // Yellow
    strip.show();
    delay(250);
}

void setAllPixels(int red, int green, int blue) {
    for (int i = 0; i < NUM_PIXELS; i++) {
        strip.setPixelColor(i, strip.Color(red, green, blue));
    }
}

// ----- Buzzer -----
void updateBuzzer(int obstacleAlert) {
    if (obstacleAlert >= 0) {
        tone(buzzerPin, 1000);
        delay(100);
        noTone(buzzerPin);
    } else {
        noTone(buzzerPin);
    }
}

// ----- Serial Monitor -----
void printSerialStatus(int occupiedCount, int availableCount, int obstacleAlert) {
    Serial.println("-----");
    Serial.println("LU Smart Parking Status");

    for (int i = 0; i < totalSlots; i++) {
        Serial.print("Slot ");
        Serial.print(i + 1);

        if (i == 3) {
            Serial.print(" Disabled");
        } else {
            Serial.print(" General");
        }
    }
}
```

```

    Serial.print(" | Distance: ");
    Serial.print(slotDistance[i]);
    Serial.print(" cm | ");

    if (slotOccupied[i]) {
        Serial.println("Occupied");
    } else {
        Serial.println("Available");
    }
}
Serial.print("Occupied Slots: ");
Serial.println(occupiedCount);

Serial.print("Available Slots: ");
Serial.println(availableCount);

Serial.print("Parking Status: ");
if (availableCount > 0) {
    Serial.println("AVAILABLE");
} else {
    Serial.println("FULL");
}

Serial.print("LEFT Obstacle: ");
Serial.print(obstacleDistance[0]);
Serial.println(" cm");

Serial.print("FRONT Obstacle: ");
Serial.print(obstacleDistance[1]);
Serial.println(" cm");

Serial.print("RIGHT Obstacle: ");
Serial.print(obstacleDistance[2]);
Serial.println(" cm");

if (obstacleAlert == 0) {
    Serial.println("Obstacle Alert: LEFT");
} else if (obstacleAlert == 1) {
    Serial.println("Obstacle Alert: FRONT");
} else if (obstacleAlert == 2) {
    Serial.println("Obstacle Alert: RIGHT");
} else {
    Serial.println("Obstacle Alert: NONE");
}

Serial.println("-----");
}

```

3.2. System Results:

- **Project results/Outputs (all methods/functions):** Following are the screenshots taken from different states of the prototype.

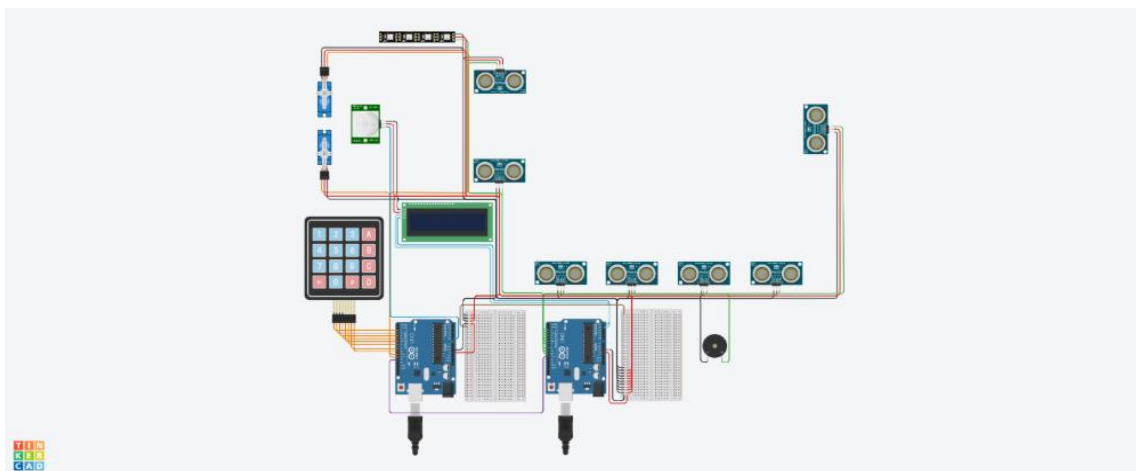


Figure 3.1: Full Tinkercad circuit design of the smart parking prototype.

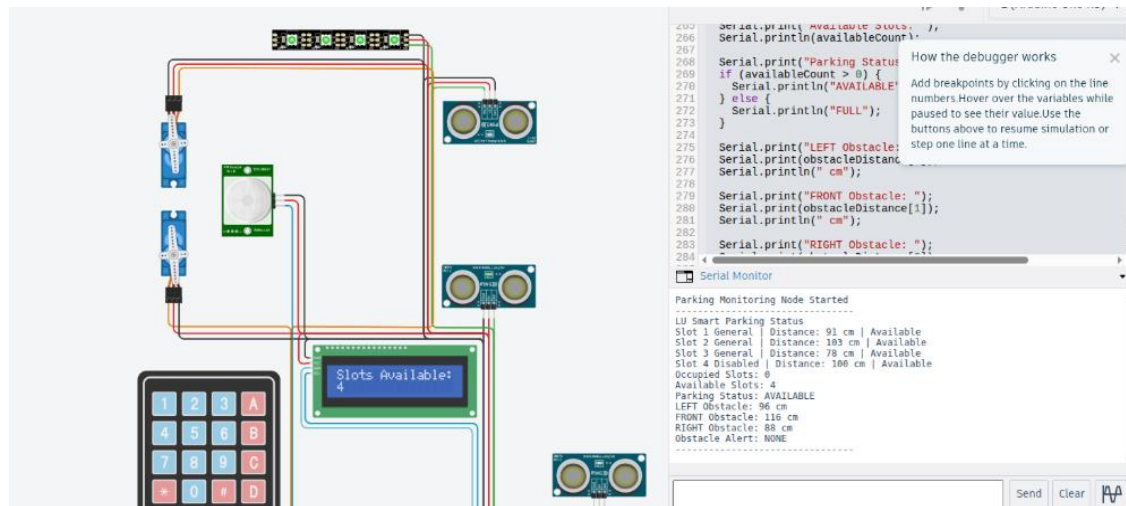


Figure 3.2: System initialization state showing LCD, NeoPixel, and Serial Monitor startup.

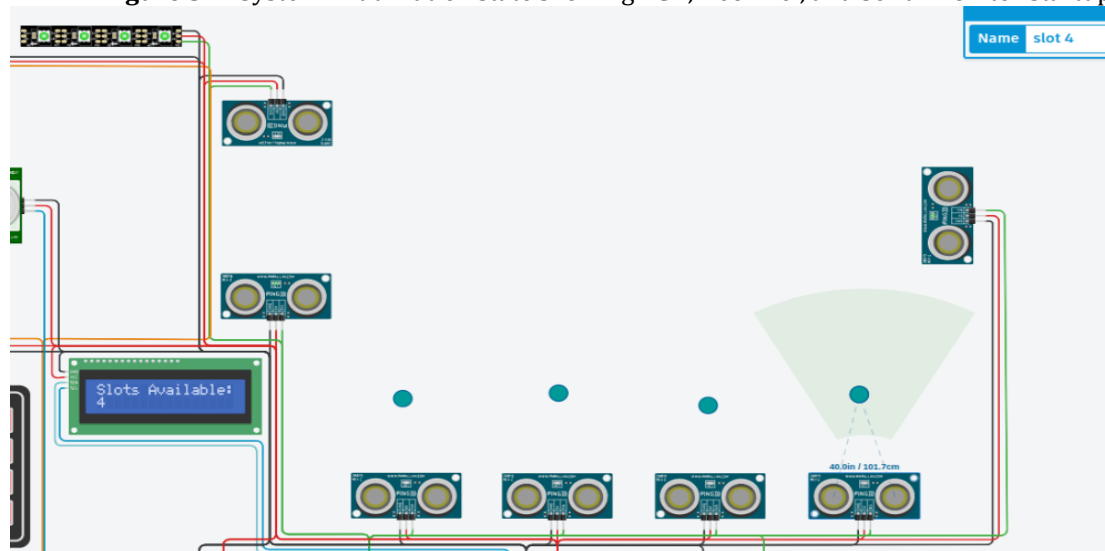


Figure 3.3: Empty parking state showing all four slots available.

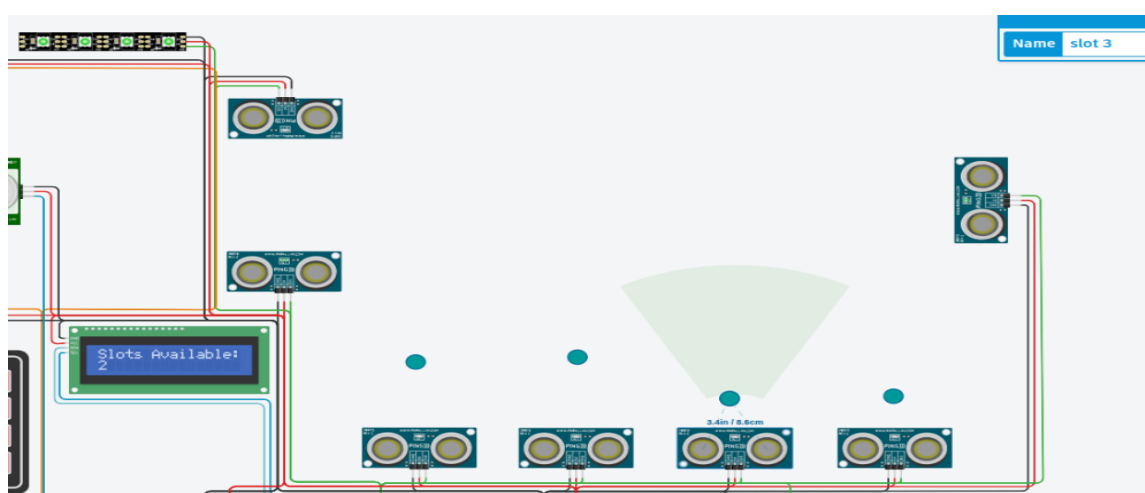


Figure 3.4: Partially occupied parking state showing updated available slot count.

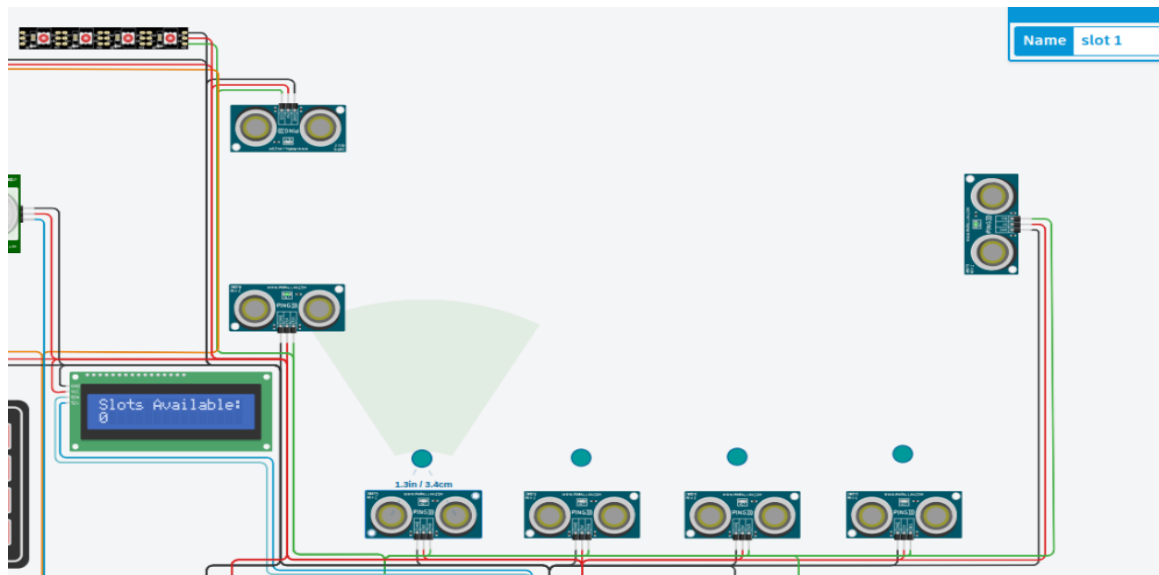


Figure 3.5: Parking full state showing zero available slots and full-status indication.

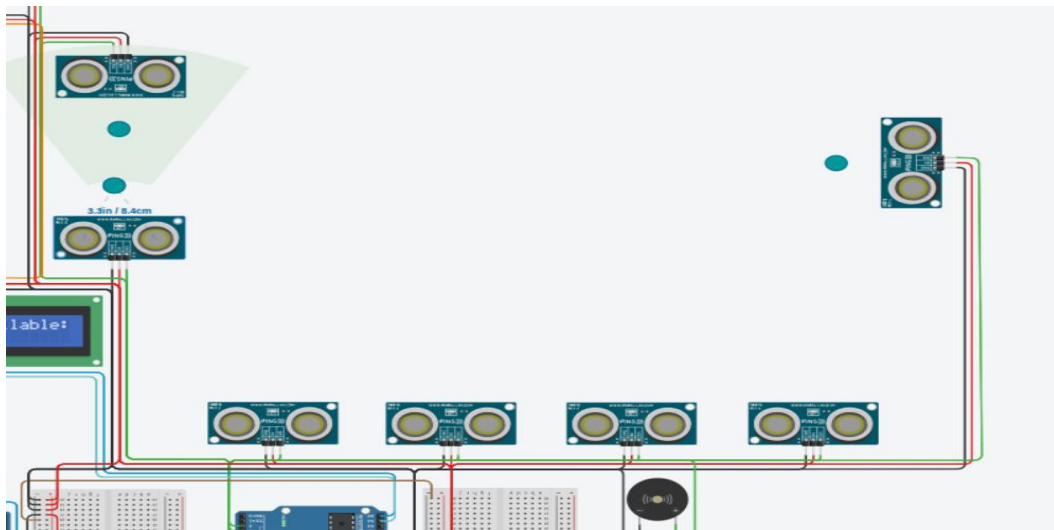


Figure 3.6: Obstacle alert state showing left, front, or right ultrasonic detection and buzzer activation.

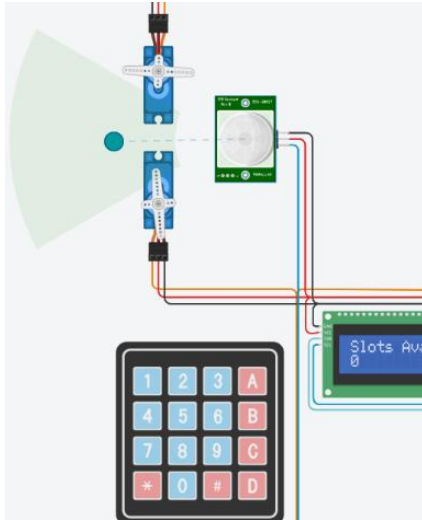


Figure 3.7: Valid entry access state after entering A1234# and triggering PIR movement.

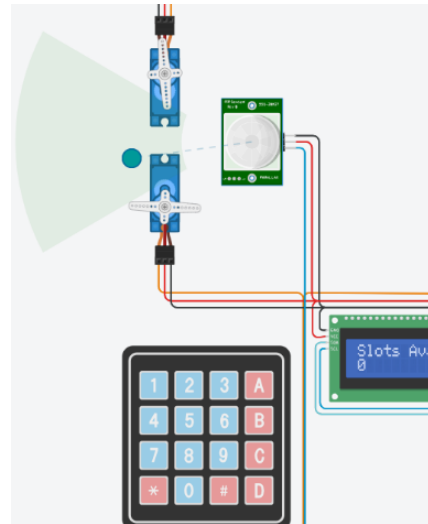


Figure 3.8: Valid exit access state after entering B1234# and triggering PIR movement.

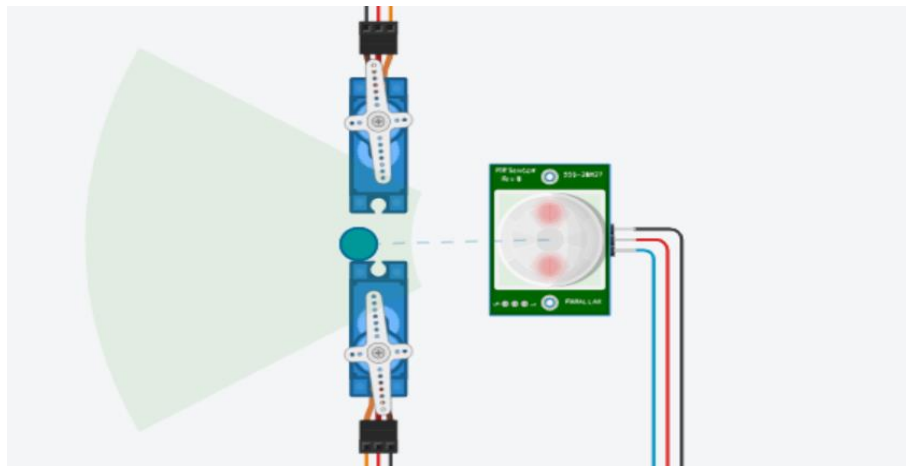


Figure 3.9: Access denied state after wrong password or invalid keypad command.

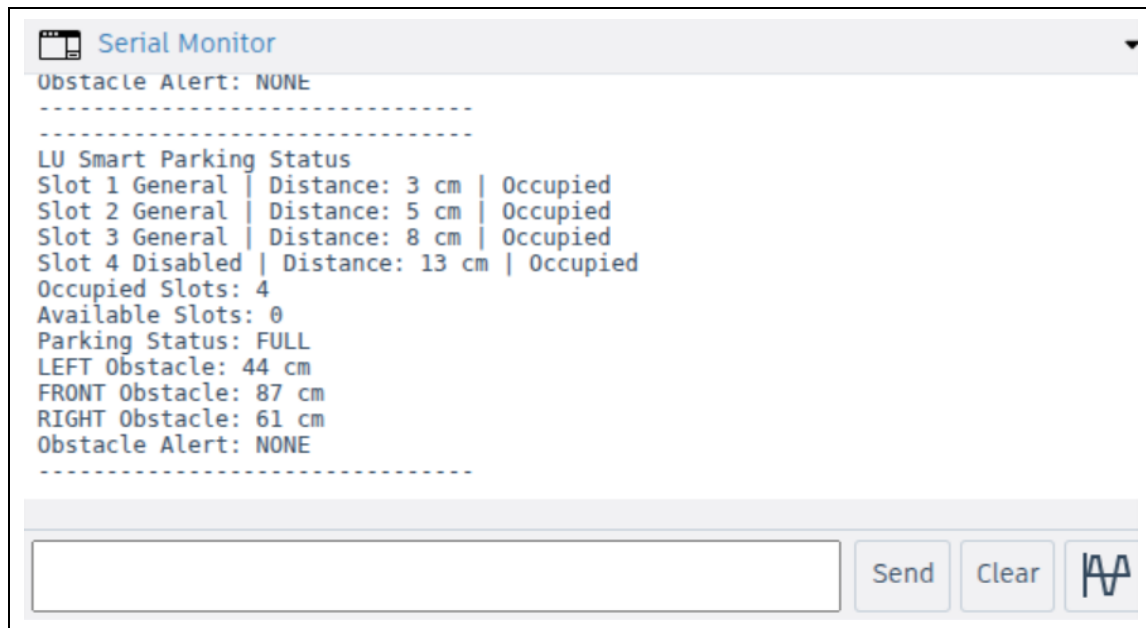


Figure 3.10: Serial Monitor output showing slot distances, availability, and obstacle status.

The test cases below describe how the system should be validated in Tinkercad. The final screenshots should match these states as closely as possible.

Test Case	Input / Simulation Action	Expected Result
T1 - System start	Start Tinkercad simulation	LCD shows welcome message and then available count, NeoPixel starts green, Serial Monitor starts.
T2 - Empty parking	Move all slot objects outside 15 cm threshold	Available count becomes 4; slots show available in Serial Monitor.
T3 - One slot occupied	Place object within 15 cm of Slot 1 sensor	Available count becomes 3; Slot 1 printed as occupied.
T4 - Parking full	Place objects within 15 cm of all four slot sensors	Available count becomes 0; NeoPixel turns red; Serial Monitor prints FULL.
T5 - Left obstacle alert	Place object within 12 cm of left obstacle sensor	Buzzer activates briefly; Serial Monitor prints LEFT alert.
T6 - Front obstacle alert	Place object within 12 cm of front obstacle sensor	Buzzer activates briefly; Serial Monitor prints FRONT alert.
T7 - Right obstacle alert	Place object within 12 cm of right obstacle sensor	Buzzer activates briefly; Serial Monitor prints RIGHT alert.
T8 - Valid entry	Enter A1234# and trigger PIR motion	Entrance servo opens to 90 degrees; gate signal goes HIGH; then closes after PIR clear.
T9 - Valid exit	Enter B1234# and trigger PIR motion	Exit servo opens to 90 degrees; gate signal goes HIGH; then closes after PIR clear.
T10 - Wrong password	Enter A9999# or B9999#	Access denied message is printed; gate remains closed.
T11 - Timeout	Enter correct command but do not trigger PIR for 60 seconds	Unlock expires and gate remains locked.
T12 - Invalid command	Enter C1234# or short input	Access denied message is printed with invalid command reason.

- **System Results/Output Constraints/Parameters:**

i). Operational/Transitional Efficiency: The system improves operational efficiency by automating three activities that would otherwise require manual checking: parking-slot monitoring, obstacle alerting, and gate control. The parking monitoring node checks all four parking slots and three obstacle sensors repeatedly, while the gate access node independently manages authentication and gate movement. Separating the two tasks improves clarity and avoids overloading one Arduino with all I/O responsibilities. The gate transition logic is also controlled carefully. A valid keypad command opens a one-minute unlock window,

and the servo only opens when the PIR sensor detects movement. The gate does not close immediately; it waits until the PIR sensor remains clear for 0.8 seconds. This reduces the risk of closing the gate while the simulated vehicle is still near the gate.

ii).

Risk

Assessment:

Risk	Possible Effect	Mitigation
Ultrasonic reading error	Incorrect slot or obstacle status	Use calibrated thresholds, ignore zero timeout readings, and test repeatedly.
PIR false trigger	Gate may open or remain open at wrong time	Require password unlock before PIR can open gate; close only after clear confirmation.
Wrong or shared password	Unauthorized gate access	Use stronger/dynamic credentials in future version.
Servo power instability	Gate movement may fail in real hardware	Use suitable external servo power and common ground in real deployment.
Wiring complexity	Misconnection or debugging difficulty	Use two-Arduino separation, documented pin map, and consistent wire colors.
Tinkercad limitation	No real cloud connectivity or weather effects	Use Serial Monitor as simulated dashboard and discuss cloud extension.

iii). System Reliability and Downtime: The prototype is reliable within the limits of a simulation because the logic uses simple sensor thresholds, deterministic servo angles, and repeated status updates. The use of a timeout in the ultrasonic pulse measurement prevents the system from waiting indefinitely if a sensor does not return an echo. Similarly, the unlock timeout prevents the gate from staying in an unlocked state forever.

In a physical deployment, downtime could occur because of sensor failure, broken wires, power interruption, moisture, snow, incorrect mounting angle, or mechanical gate obstruction. A production version should include weatherproof enclosures, health-check messages, backup power, watchdog timers, cloud error logging, and a manual override switch.

iv). Data Security & Privacy: The prototype does not collect personal identity data. The password is fixed in the Arduino sketch, and access events are printed to Serial Monitor as simulated JSON-style payloads. Because no real internet connection is used, the simulation has low privacy risk. However, a real smart parking system would require stronger security controls. Fixed passwords should be replaced with dynamic credentials, RFID cards, encrypted authentication, license plate recognition, or mobile app login. If a cloud dashboard is added, data transmission should use secure channels, and access logs should be minimized to only what is necessary for parking management.

3.3. Conclusion:

- **Conclusion:** The IoT-based smart parking system demonstrates how Arduino and Tinkercad can be used to model a smart-city parking solution. The prototype monitors four parking spaces, detects left/front/right obstacles, activates buzzer alerts, controls entrance and exit gates with servo motors, displays availability on an LCD, uses NeoPixel status lighting, and prints simulated IoT dashboard data through Serial Monitor.

The project successfully connects IoT concepts with a practical smart-city problem. It shows the complete cycle of sensing, processing, decision-making, and automated response. While the current design is simulation-based, it provides a strong foundation for a physical prototype or future cloud-connected system.

- **Future Enhancements:** There are several flaws in the current prototype which we identified over the period of data collection and documentation we've identified the following issues with the current design, and we would prefer to list those as future enhancement.
 - Replace PIR sensors with an ultrasonic distance sensor, since in case a vehicle stands still under the gate, the PIR will sense no motion, and the servo gate will end up colliding with the vehicle.

- With the entrance and exit gate control, we're already able to count the vehicles. Ultrasonic distance sensors in parking are technically not needed for parking availability calculation.
- An additional LCD can make the experience of password entry at the gate.
- The NeoPixel strip is practically extra power consumption, not adding any extra value to the over all system.

Above are the improvements that can be made in the Tinkercad environment. Following, we have prepared a list of enhancements that can improve this circuit to install on a physical smart parking lot in LU.

- Add Wi-Fi connectivity using ESP8266 or ESP32 to send status to a cloud dashboard.
- Create a mobile app for checking parking availability before arrival.
- Add RFID or QR-based authentication instead of a fixed keypad password.
- Use automatic number plate recognition for secure entry and exit.
- Add database storage for parking history, usage analysis, and maintenance logs.
- Add online booking and reservation features.
- Add solar power or battery backup for outdoor sensors.
- Use weatherproof hardware and physical mounting brackets for real deployment.
- Add fault detection when a sensor returns invalid values repeatedly.
- Add separate LEDs for each parking slot to improve visual debugging.

References

- [1] IBM, "What is the Internet of Things (IoT)?" IBM Think. <https://www.ibm.com/think/topics/internet-of-things>
- [2] Y. Geng and C. G. Cassandras, "New Smart Parking System Based on Resource Allocation and Reservations," IEEE Transactions on Intelligent Transportation Systems, 2013. https://people.bu.edu/cgc/Published/TITSprint_09_13.pdf
- [3] A. O. Elfaki et al., "A Smart Real-Time Parking Control and Monitoring System," Sensors, 2023. <https://www.mdpi.com/1424-8220/23/24/9741>
- [4] X. Chen, Z. Qian, R. Rajagopal, T. Stiers, C. Flores, R. Kavalier, and F. Williams III, "Parking Sensing and Information System: Sensors, Deployment, and Evaluation," 2017. <https://arxiv.org/abs/1712.02741>
- [5] Autodesk Tinkercad, "Official Guide to Tinkercad Circuits." <https://www.tinkercad.com/blog/official-guide-to-tinkercad-circuits>
- [6] Arduino, "UNO R3 Documentation." <https://docs.arduino.cc/hardware/uno-rev3>
- [7] Parallax Learn, "(PING))) Ultrasonic Distance Sensor." <https://learn.parallax.com/kickstarts/ping-ultrasonic-distance-sensor/>
- [8] Arduino, "pulseIn() Language Reference." <https://docs.arduino.cc/language-reference/en/functions/advanced-io/pulseIn/>
- [9] Arduino, "Servo Library Documentation." <https://www.arduino.cc/en/reference/servo>
- [10] Arduino, "Keypad Library Documentation." <https://docs.arduino.cc/libraries/keypad/>
- [11] Arduino, "LiquidCrystal Library Documentation." <https://docs.arduino.cc/libraries/liquidcrystal>
- [12] Adafruit, "Arduino Library Use - Adafruit NeoPixel Uberguide." <https://learn.adafruit.com/adafruit-neopixel-uberguide/arduino-library-use>
- [13] Adafruit, "Adafruit_LiquidCrystal Library." https://github.com/adafruit/Adafruit_LiquidCrystal